Bản nộp gồm báo cáo chính và 3 trang phụ lục minh chứng ở cuối. Nguồn/artifact nội bộ được chuyển thành hyperlink GitHub theo commit cố định `f2becc501cfc`. Không cần đường dẫn trên máy xuất PDF để mở bằng chứng.

So sánh OpenCode, Pi và Codex CLI; OpenCode v1 và v2

Bản tổng hợp hoàn tất ngày 2026-11-11. Benchmark OpenAI trực tiếp đã chạy **48/48 lượt hợp lệ**, đạt **46 success, 2 failure, không timeout/infra error**. Cả bốn CLI qua native smoke thật; snapshot của cả 48 lượt qua scope/public/hidden tests. Hai failure vẫn được giữ vì native error, không bị nâng thành success chỉ do snapshot đúng.

Đọc nhanh: phương pháp, so sánh ba tool, v1/v2, kết quả benchmark, hạn chế, khuyến nghị, bằng chứng đi kèm. Đã có đầu ra cho 8/8 hạng mục trong phạm vi nghiệm thu; không có nghĩa mọi tính năng đều pass hoặc không còn giới hạn nghiên cứu.

Ngày chốt phiên bản: 06/11/2026. Stable release được đối chiếu upstream tại mốc này và giữ nguyên suốt nghiên cứu; không có nghĩa vẫn mới nhất ngày nộp. Metadata và full SHA.

Lịch sử provider, sửa harness và protocol cũ ở EXPERIMENT_HISTORY.md; không nhập vào cohort cuối.

1. Tóm tắt kết quả

Cấu hình	End-to-end success	Snapshot pass	Median thời gian/lượt	Guard estimate cho 12 lượt
OpenCode v1.19.5	11/12	12/12	228,15 giây	$0,49
OpenCode v2.2.5	12/12	12/12	200,20 giây	$0,42
Pi v1.5.0	11/12	12/12	164,23 giây	$0,26
Codex rust-v0.167.0	12/12	12/12	139,63 giây	$0,97

Tổng chi phí được tính từ số liệu gốc trước khi làm tròn từng công cụ; tổng các giá trị hiển thị có thể lệch $0,01 so với tổng chung.

- **Cohort chính:** OpenAI trực tiếp, `gpt-6.1-sol`, 4 task × 4 CLI × 3 lượt. Benchmark mất **2 giờ 37 phút 49 giây**; 278/278 upstream HTTP 200/completed, không retry để nâng điểm. Median gồm failure, startup/teardown và pacing ≥ 30 giây, không phải model latency thuần.
- **Chi phí:** guard estimate benchmark khoảng **$2,15**, gồm mức cache-write bảo thủ khi thiếu TTL; không phải hóa đơn. Không có request unpriced mới trong cohort, nhưng sáu request unpriced lịch sử vẫn unknown. Smoke bốn CLI r3 tách riêng, khoảng $0,10. Số chính xác giữ trong artifact, không tăng độ chắc chắn của ước tính.
- **Ý nghĩa:** Codex có median thấp nhất và ít upstream nhất, Pi có estimate thấp nhất trong cohort. v2/Codex đạt 12/12, v1/Pi đạt 11/12; n = 3/task và native prompt/tool schema khác, không suy thứ hạng tổng quát hoặc quan hệ nhân quả từ bản CLI.
- **Bằng chứng:** 147 evidence IDs theo commit cố định, tách source/runtime. Demo native permission, Client/SDK, plugin port, background/compaction plumbing, managed recovery/budget và shared UI có acceptance riêng; inference scripted local, không phải phép đo model quality. Vòng 1, vòng 2 và giới hạn.
- **Tái lập:** protocol, toàn bộ 48 lượt và raw, audit cuối. Không trộn với pilot Shopaikey hoặc smoke Google/Gemini.

2. Phạm vi và phương pháp

- Phiên bản/full SHA chốt ngày 06/11/2026: research/versions.md; không cập nhật pin giữa thực nghiệm hoặc thay service người dùng.
- OpenCode v2 đại diện OpenCode trong so sánh ba tool; bốn cấu hình thực nghiệm gồm cả v1.
- Provider/model của benchmark hoàn chỉnh: `https://api.openai.com/v1`, `gpt-6.1-sol`, Responses SSE; `effort=low`, `tier=default`, `store=false` và giữ native reasoning fields. Shopaikey alias cùng tên không chứng minh cùng backend; các cohort cũ tách riêng.
- Giá OpenAI published standard short context dùng cho estimate/guard, không invoice-confirmed; cache-write thiếu TTL dùng conservative 12h upper bound, không cộng chồng ordinary input. Giá Shopaikey user-reported chỉ áp cohort cũ. Tách **source-confirmed / runtime-confirmed / both / unverified** và diagnostic/pilot/smoke/benchmark.
- Không sửa source repos/binary/service đang dùng, không nạp session thật hoặc hiển thị key. Benchmark được duyệt dùng credential có quyền file `600` trong host memory; CLI chỉ nhận local proxy token. Config/session/workspace mới mỗi lượt, broker là đường provider duy nhất.

Nguồn metadata và API offline: preparation, HTTP offline. Source claims ở evidence.csv không tự chứng minh runtime với model/provider bất kỳ.

Demo native bổ sung: acceptance matrix và kết quả vòng2 tách fixture unit tests khỏi native runtime. Permission CLI/API, Client/SDK, plugin port, background, compaction plumbing, managed-service recovery và shared UI đều có acceptance riêng, 0 API thật mới. Chưa chứng minh model quality/semantic retention hoặc exactly-once.

3. So sánh ba tool

Các mục 3.1–3.6 là **đối chiếu source tại release đã pin**, kèm trạng thái demo cuối cùng khi có; không phải proof mọi mode của sản phẩm. Pi/Codex đọc bằng `git show <release-SHA>:<file>`, không dùng HEAD development thay release. Demo OpenCode có phạm vi riêng; chất lượng context, native OS security và compatibility ngoài marker plugin vẫn chưa được chứng minh.

Trục	OpenCode v2	Pi	Codex CLI
Kiến trúc	Client/service, standalone và embedded host	Ghép Agent với coding AgentSession	Nhánh exec dùng in-process app-server client
Agent loop	Điều phối step/attempt và continuation	Stream → tool results → steering/follow-up	Turn, pending input và in-flight tool futures
Tool	Schema → runtime decode/execute/normalize	Tool factories và session allowlist	Router → registry; parallel capability theo tool
Permission	Ordered rules, saved approval, enforcement	Callback/extension có thể chặn tool	Approval policy tách sandbox backend
Context	History projection theo boundary/model	Active branch path và compaction entries	Model items và retained context riêng
Mở rộng	Plugin domains, tool transform, cleanup	Event handlers, registerTool/registerCommand	Plugin manifest với skills/MCP và các field khác

Bảng định hướng được rút từ source permalinks ở 3.1–3.6, không bổ sung kết luận runtime. **Trade-off:** OpenCode cần chọn đúng hosting/lifecycle khi tích hợp nhiều client; Pi cho phép phân biệt tùy biến vòng lặp với quản lý coding session; Codex yêu cầu phân biệt turn/tool routing với approval/sandbox. Các ranh giới này quyết định nơi gắn custom tool, nơi chờ completion và nơi enforce policy, không tự chứng minh tool nào nhanh hoặc an toàn hơn.

3.1. Kiến trúc

Tool	Đường thực thi đã truy vết	Hệ quả khi sử dụng và giới hạn
OpenCode v2	`ServerConnection.resolve` ưu tiên explicit server, sau đó standalone/disabled-service, cuối cùng managed service. `Standalone.start` spawn private `serve --stdio --port 0`; embedded SDK còn có đường `EmbeddedHost` + `host.fetch`. E-0014, E-0015, E-0053.	Phải chỉ rõ đang dùng managed service, private child hay embedded host; không gộp chúng thành một lifecycle. Standalone không tự cách ly config/data. Shared TUI/Desktop/Web đã kiểm trong Linux headless sandbox, không suy mọi hosting mode (mục 4.1).
Pi	Trong `createAgentSession`, SDK dựng `Agent` với transcript hiện có và stream qua `modelRuntime`, rồi ghép `AgentSession` với session/settings/resource managers. E-0077.	Phân biệt lớp vòng lặp `Agent` với lớp coding-session quản lý môi trường/phiên. Đây là đường ghép SDK được đọc, không kết luận Pi không có server/mode khác hoặc đã nghiệm thu SDK runtime.
Codex CLI	Nhánh `exec` kiểm Git trust rồi khởi tạo `InProcessAppServerClient`. E-0028.	`exec` cũng có lớp app-server client; không nên mô tả là chỉ gọi model trực tiếp hoặc mặc nhiên bắt buộc remote server. Kết luận giới hạn ở nhánh `exec`, không phủ mọi TUI/daemon mode.

So sánh thực dụng: xác định ranh giới client/server/session trước khi chọn cách tích hợp hay đo startup. Các đường trên khác cách ghép thành phần; source chưa cho phép kết luận tool nào nhanh/nhẹ/ổn định hơn.

3.2. Vòng lặp agent

Tool	Quyết định tiếp tục ở đâu?	Bằng chứng
OpenCode v2	`drain` lấy step sẵn sàng rồi gọi `runStep`; `runStep` kiểm compaction, chuẩn bị primary model request, gọi `steps.attempt`. Outcome `Completed` còn có `needsContinuation`, còn `Retry` đi qua retry policy/wait.	E-0080: drain, E-0081: runStep.
Pi	`runLoop` stream assistant, thực thi tool calls và đưa tool results vào context. Error/aborted kết thúc nhánh; steering/follow-up và explicit continuation có thể giữ vòng lặp hoạt động sau một response.	E-0078: runLoop.
Codex CLI	`run_turn` nhận kết quả sampling rồi hợp `model_needs_follow_up` với pending input. Nhánh còn in-flight tool futures chờ `drain_in_flight` trước token-count event tương ứng.	E-0083: run_turn, E-0086: drain tools.

Hệ quả cho automation: “provider đã hoàn tất một response”, “tool đã xong”, “agent/CLI đã kết thúc” và “acceptance đã đạt” là các mốc khác nhau. Vì vậy benchmark lưu terminal provider, CLI exit và supervisor grading riêng; không nâng snapshot pass thành success khi CLI chưa hoàn tất. Chưa có đối chứng runtime riêng cho mọi nhánh steering/compaction/retry, không dùng source flow để khẳng định retry tốt hơn hoặc tiết kiệm token hơn.

3.3. Hệ thống tool

Tool	Cách tổ chức đã xác minh từ code	Hệ quả và giới hạn
OpenCode v2	`definition` dựng model tool definition từ schema; `execute` đi qua input decode, tool implementation, output encode/normalize và chuẩn hóa foreign typed failure về `Tool.Error`. E-0082.	Có ranh giới hợp đồng tool/implementation; cần đọc phần validation cụ thể trước khi kết luận mọi loại JSON Schema đều được enforce. Đây không phải permission engine hoặc bằng chứng OS sandbox.
Pi	`createCodingTools` tạo `read/bash/edit/write`; `createReadOnlyTools` tạo `read/grep/find/ls`; `createAllTools` là factory tập khác. E-0079.	Không đồng nhất tập factory với inventory hiệu lực của mọi session: CLI còn parse tool allowlist. E-0068. Tên "read-only" của factory không tự giới hạn filesystem ở mức OS.
Codex CLI	`ToolRouter` tạo `ToolInvocation` mang session/step/cancellation rồi dispatch qua registry; `tool_supports_parallel` đọc capability từng tool và mặc định false nếu thiếu. E-0084, E-0085.	Capability song song là có điều kiện, không phải mọi tool luôn parallel; không thay concurrency 1 của thí nghiệm. Cần đọc handler/policy để đánh giá quyền và sandbox, không suy từ router.

Không so tool bằng số phần tử `tools` top-level đơn thuần: nhánh Codex Responses Lite có thể đưa `additional_tools` vào `input` và đặt header tương ứng. E-0060, E-0062. Tool definitions, runtime dispatch và OS permissions phải được ghi riêng; chưa có bảng hiệu năng tool-call đồng nhất để xếp hạng.

3.4. Permission và sandbox

Ba lớp cần phân biệt: **cho phép gọi tool**, **hỏi/phê duyệt hành động**, và **hạn chế tài nguyên ở mức OS**. Không dùng tool allowlist, config home hoặc HTTP authentication thay cho OS sandbox.

Tool	Đường kiểm quyền đã truy vết	Điều chưa được chứng minh
OpenCode v2	Shell prepare resolve cwd/directories, `authorizeExternal`, rồi `permission.assert` trên parsed commands. Evaluator chọn rule khớp cuối; enforcement còn kiểm configured deny trước saved allows và đi qua hook/unattended policy. E-0089, E-0087, E-0088.	Có application-level policy không đồng nghĩa parser chặn mọi shell side effect hoặc tự cung cấp kernel sandbox. Allow/deny, API approval và saved scope đã kiểm ở muc 4.3; chưa kiểm keyboard UI hoặc durable approval qua restart.
Pi	Local shell operations mặc định gọi `spawn` với cwd/env và timeout/abort cleanup. Tool preparation có `beforeToolCall`; nếu callback trả `block`, tạo immediate error thay vì prepared call. E-0090, E-0092.	Đây là local helper/hook path, không đủ kết luận Pi không có custom isolation hoặc mọi mode đều có approval mặc định. Hook chặn tool không phải OS-level filesystem/network confinement.
Codex CLI	Protocol tách `AskForApproval` khỏi `SandboxType`; approval có `Never`, backend type inventory có None/LinuxSeccomp/MacosSeatbelt/WindowsRestrictedToken/WindowsMxc. E-0093, E-0094.	`Never` nghĩa là không hỏi, không có nghĩa tắt sandbox. Enum không chứng minh backend nào thật sự active, mức deny nào được enforce hay tương đương trên các OS.

Pi extension có đường can thiệp trước tool thật, không chỉ declaration: `AgentSession` cài callback vào Agent (E-0107); `createLoopConfig` chuyển callback cho vòng lặp (E-0108); extension emitter trả ngay khi handler báo block (E-0091). Đây vẫn là source wiring, chưa chạy extension demo.

Hệ quả: phải chọn approval policy và isolation policy riêng. Bwrap/read-only mounts của harness thí nghiệm là lớp ngoài do nghiên cứu đặt, không được dùng làm bằng chứng ba tool có native sandbox tương đương. Chưa đủ dữ liệu để xếp hạng bảo mật hoặc kết luận một tool "an toàn nhất".

3.5. Context và session

Phân biệt **history đã giữ**, **projection đưa vào model**, **checkpoint/summary**, và **execution đang chạy**. Chúng không đồng nghĩa chỉ vì cùng mang tên session/context.

Tool	Cơ chế source đã xác minh	Hệ quả và giới hạn
OpenCode v2	History phân biệt native compaction window và các boundary `latest`/`local`/model ref. Helper `replayable` kiểm providerID/model id và khả năng decode khi dùng model boundary; local boundary không replay native window. E-0095.	Projection phụ thuộc boundary/model, không suy summary/window dùng được lossless qua mọi model. Native compaction plumbing đã kiểm bằng scripted summary (E-0134/kết quả vòng 1); chất lượng giữ ngữ nghĩa sau nén vẫn unverified.
Pi	`buildContextEntries` theo active leaf path, chọn latest compaction và giữ range từ `firstKeptEntryId` cùng các entry sau compaction. `branch` chỉ xác minh entry và chuyển leaf pointer trong hàm đó. E-0096, E-0097.	Branch selection và model context là projection của history, không phải xóa mọi entry cũ. Chưa demo branch/resume CLI hoặc bảo đảm tiếp tục tool execution đang chạy.
Codex CLI	`ContextManager` giữ model items và retained context riêng; `conversation_history_snapshot` ghép shared items/review history/retained context/reviewer mode/revisions. E-0098.	In-memory snapshot không tự chứng minh disk persistence, crash recovery hoặc khôi phục process/tool. Chưa đo chất lượng giữ ngữ cảnh hoặc token saving so với tool khác.

Do đó chưa thể kết luận tool nào "nhớ tốt hơn" từ data model. Mục 4.2 có cả metadata qua restart và demo Client giữ transcript trong request tiếp theo; compaction plumbing có scripted summary riêng. Các kết quả này không chứng minh semantic retention hoặc khả năng đổi model lossless. Không dùng số token/cache của một lượt để thay phép thử chất lượng context.

3.6. Khả năng mở rộng

Tool	Bề mặt mở rộng tại pin	Giới hạn khi chọn/cải biến tích hợp
OpenCode v2	Promise plugin có `id/setup(context)` và Cleanup; context cung cấp các domain. Tool domain có transform/reload/list/hook, editor add/update/remove. E-0099, E-0100.	Marker plugin đã port, gọi tool và quan sát cleanup ở mục 4.5. Chưa nghiệm thu toàn bộ hook ordering/reload hoặc compatibility của plugin khác.
Pi	Extension API có `tool_call/tool_result/user_bash` handlers và `registerTool/registerCommand`. E-0101.	Có thể thiết kế tích hợp theo event/registration contract này; chưa chạy extension để chứng minh behavior hoặc isolation. Không mặc nhiên coi API giống OpenCode plugin.
Codex CLI	`RawPluginManifest` có fields skills/mcp_servers/apps/hooks/interface/extensions. E-0102.	Đây là schema của manifest được đọc; không đủ suy mọi field được kích hoạt, không tương đương một JavaScript plugin function và không kết luận không có extension mechanism khác.

Khuyến nghị: chọn theo bề mặt cần mở rộng (tool/event/lifecycle hay manifest/skills/MCP), rồi kiểm loader/config/trust và runtime riêng. Tên "plugin" chung không phải compatibility contract giữa sản phẩm. OpenCode marker plugin đã kiểm runtime trong sandbox; Pi extension và Codex plugin ở đây vẫn là đối chiếu source, không bị kết luận unsupported chỉ vì chưa demo.

4. OpenCode v1 so với v2

4.1. Kiến trúc và background service

- **v1 không mặc định đồng nghĩa "không có server":** nhánh `run` không attach tạo SDK custom fetch gọi `Server.Default().app.fetch` trong process; đồng thời có `ServeCommand` mở listener và giữ process. E-0008, E-0044.
- **v2 tách đường kết nối rõ:** explicit server, private standalone child và managed-service resolution là các nhánh riêng; embedded SDK còn có host riêng. ServerConnection.resolve/E-0014, Standalone.start/E-0015, PromiseSdk.create/E-0053.

Khác biệt chính là **cách resolve/host server và quản lý lifecycle**, không phải "v1 không server/v2 luôn daemon". Trường hợp các client cùng kết nối managed service:

```
TUI ---------+
Desktop -----+--> Private managed service
Web ---------+      ├── Sessions / history
                    ├── HTTP API / events
                    └── Agent / tools / permissions

Standalone --> Private child server   (lifecycle khác)
Embedded SDK -> In-process host       (không HTTP listener)
```

Runtime-confirmed, E-0142/E-0143: UI05 tạo session qua API, mở cùng ID/title trong Desktop 2.2.5, web frontend do service phục vụ và TUI. Desktop/TUI dùng default discovery, PID service đầu/cuối không đổi. Desktop chạy headless; web render trong Chromium engine của package, không phải browser độc lập; TUI chạy trong PTY. Có ảnh Desktop, ảnh Web, raw PTY và kết quả kiểm ID/PID. Không tác động display/service người dùng hoặc suy rộng sang mọi OS/hosting mode. Quy trình và giới hạn.

Ảnh chụp bổ sung sau review phần 4.1: lượt UI cách ly mới có Desktop với khung cửa sổ, Chrome độc lập thấy thanh địa chỉ và TUI trong xterm. Cùng session `ses_e34ef23f6ffek2g8YsPzUztvXw`, service PID `12 → 12`; review raw/ảnh. Đây là lượt mới không gửi prompt, không sửa kết quả UI05 hoặc benchmark; ảnh mới đưa vào phụ lục PDF.

4.2. API/Client/SDK và demo

Mặt tích hợp	Bằng chứng	Trạng thái và giới hạn
HTTP create session v1/v2	v1 `POST /session` trả trực tiếp `Session.Info`; v2 `POST /api/session` nhận `location` và trả envelope `data`. E-0042 v1, E-0041 v2.	**both** cho nhánh create session rỗng của demo; không coi đổi prefix là đủ để migrate toàn API.
HTTP auth/events — metadata	Event v1/E-0046, event v2/E-0045, auth v1/E-0048, auth v2/E-0047.	**both** cho auth/SSE/session ID trong HTTP offline demo. Prompt/events và permission là các case riêng bên dưới, không suy replay/automatic reconnect.
Session metadata qua restart v1/v2	E-0050 v1, E-0051 v2.	**runtime-confirmed** cho ID/title session rỗng trong HTTP offline demo. V2 transcript qua restart được kiểm riêng bằng Client; crash recovery ở 4.4.

Mặt tích hợp	Bằng chứng	Trạng thái và giới hạn
TypeScript Client v2	Package exports tách Promise/Effect client và service entrypoints; `Service.discover` khác `ensure` có nhánh spawn. E-0052, E-0054.	**both** trong phạm vi Client source đúng pin: prompt/events/transcript qua private server restart (E-0125/raw). Không suy mọi nhánh `Service` lifecycle đều đã kiểm.
Embedded SDK v2	`PromiseSdk.create` ghép `EmbeddedHost` với client dùng `host.fetch`, có aliases sessions/events và close/disposal. E-0053.	Contract **source-confirmed**; published SDK 2.2.5 **runtime-confirmed** cho prompt/events/close (E-0128/raw). Exact package-build → source commit chưa attested; không gộp embedded host với network Client/shared service.

Runtime, E-0125/E-0127/E-0128: Client nhận prompt/events/transcript và history sau private server restart. Location-shell API kiểm running/exit 7/cancel không giết sibling và observer mới sau disconnect. Embedded SDK 2.2.5 kiểm prompt/events/explicit close với cùng host PID, `urls=[]`, nhưng `server.info.version=unknown`; chưa attested exact build → commit. Full installed-tree hash không phải lockfile. Failure matrix kiểm terminal failed khi scripted provider trả HTTP 400, wait timeout không tự hủy execution, explicit interrupt và Client thiếu auth trả 401. Không suy mọi failure mode đã phủ.

4.2.1. Demo tạo session, gửi prompt và nhận event

Code thực tế: TypeScript Client runner, host launcher/cách ly. Client source được xuất bằng `git show` đúng pin; không dùng HTTP Python để thay nghiệm thu thư viện TypeScript. Luồng chạy:

```
Private server → Client kết nối → subscribe và chờ server.connected
    → tạo session → gửi prompt → nhận events
    → chờ execution kết thúc → đọc context
    → restart private server → đọc lại transcript → gửi continuation
```

Hai trích đoạn dưới lấy từ cùng runner; đây **không phải script standalone**. Launcher chuẩn bị môi trường/fixture; runner tạo Client, readiness timeout và cleanup. Đầu tiên, vòng lặp subscribe lưu từng event, cập nhật bộ đệm theo type/ID và chỉ cho phép tạo session sau `server.connected`:

```
const signal = controller.signal
void (async () => {
  for await (const event of client.event.subscribe({ signal })) {
    result.events.push(event)
    if (JSON.stringify(event).includes(marker) && !result.parent_marker_received_at)
result.parent_marker_received_at = performance.now()
    const key = eventKey(event)
    received.set(key, event)
    receivedAt.set(key, performance.now())
    if (listeners.has(key)) { listeners.get(key)!(event); listeners.delete(key) }
    if (event.type === "server.connected") ready()
    if (result.events.length > 1000) throw new Error("event cap")
  }
  if (!signal.aborted) throw new Error("event stream ended before cancellation")
})().catch((error) => {
  result.event_error = String(error)
  readinessFailed(error)
})
await connected.finally(() => clearTimeout(readinessDeadline))
```

Sau khi subscription sẵn sàng, runner tạo session/gửi prompt; `wait` chờ execution kết thúc trước khi đọc lại context:

```
const created = await client.session.create({
  title: "native-client-demo",
  location: { directory: process.cwd() },
})
await client.session.prompt({ sessionID: created.id, text: seed })
await client.experimental.session.wait({ sessionID: created.id })
const context = await client.session.context({ sessionID: created.id })
check("assistant_marker", JSON.stringify(context).includes(marker))
check("user_message", JSON.stringify(context).includes(seed))
```

Cách chạy: từ workspace, sau binary/isolation preparation và các prerequisites của runner: Linux/bwrap, Bun, Python, payload đã pin và source repo v2 tại đường dẫn trong launcher. Dùng artifact ID mới; runner từ chối ghi đè:

```
python3 -B demos/client_runtime.py \
  --artifact-dir "demos/artifacts/REVIEW-CLIENT-$(date -u +%Y%m%dT%H%M%S%N)" \
  --mode lifecycle
```

Output đã quan sát, trích trường từ raw Client, không phải log dựng hoặc lần chạy mới:

```
session.created: ses_e383b54d3ffesBFcD9WgE4b3Y8
prompt: REPORT_CLIENT_USER_SEED
assistant marker: REPORT_CLIENT_ASSISTANT_MARKER
checks.subscribed_before_prompt: true
checks.transcript_after_restart: true
checks.resume_request_contains_prior_context: true
```

Mẫu terminal event nhận được, trích nguyên ba trường từ cùng raw:

```
{
  "created": 1794352917765,
  "type": "session.execution.succeeded",
  "data": { "sessionID": "ses_e383b54d3ffesBFcD9WgE4b3Y8" }
}
```

`created` tương ứng **2026-11-10 23:21:57.765 UTC**: thời điểm server ghi event execution thành công trong demo scripted, không phải thời điểm CLI exit hoặc thời điểm client nhận gói tin. Cùng event stream còn có `session.execution.started`; `sessionID` cho phép đối chiếu với session đã tạo. Native error/interrupt có acceptance riêng, không dùng sự kiện success này thay mọi terminal outcome.

Validation: subscription sẵn sàng trước prompt; event có session tương ứng; context có user seed và assistant marker; PID đổi sau restart; request continuation chứa history trước đó. **Giới hạn:** native API/Client chạy thật nhưng inference dùng scripted local SSE, không đo model quality, automatic reconnect hoặc crash giữa stream. Managed crash recovery có case riêng ở 4.4.

Embedded SDK là demo khác: SDK runner, launcher, raw; host trong process và không HTTP listener, không phải network Client.

4.3. Permission schema, precedence và defaults

Tầng	OpenCode v1	OpenCode v2
Schema cấu hình đã truy vết	Input permission nhận action hoặc object rule, có các key như read/edit/bash/task. E-0066.	Native `Rule` có action/resource/effect, `Ruleset` là array. E-0067.

Tầng	OpenCode v1	OpenCode v2
Internal evaluator	Flatten rulesets, match permission/pattern, lấy rule cuối hoặc fallback ask; `ask` xét ruleset cùng approved, rồi deny/allow/pending deferred. E-0103.	Match action/resource, lấy rule cuối hoặc fallback ask. `configured` ghép agent permissions trước session permissions, nhưng enforcement còn nhiều bước. E-0087, E-0088.
Base agent default, chưa user overlays	Wildcard allow với exceptions ask/deny; Build ghép defaults, built-in overrides rồi user permission. E-0105.	`Agent.Info.default` cũng có wildcard allow và exceptions ask cho env files/allow env.example. E-0106.

Hai điểm dễ hiểu sai:

1. “Không rule khớp thì ask” là fallback của evaluator, **không phải** mọi thao tác mặc định đều ask: base defaults/config/agent/session có thể đã có rule khớp. Các defaults source ở bảng cũng không phải effective policy của benchmark đã override.
2. V2 không thể rút gọn toàn policy thành “rule cuối thắng” hoặc “hễ có deny là thắng”: trước hết chọn effective configured rule; nếu một resource bị configured deny thì trả deny **trước** saved allows. Sau đó mới xét saved rules, plugin evaluation và unattended settlement. Một deny cũ đã bị configured rule sau override khác với một configured deny còn hiệu lực.

Runtime: 10 CLI cases v1/v2 kiểm witness allow/deny/overlap/ask unattended; API v2 kiểm once/reject/always/pending/saved scope/deny precedence (E-0126). V1 API có sáu turn kiểm approval với witness, once hỏi lại, always theo pattern và reject (E-0140). Năm case legacy permission object trên v2 giữ behavior (E-0141); unmatched fallback ask kiểm riêng sau khi test plugin bỏ base wildcard (E-0144). Không phải keyboard UI approval, không suy durable approval qua restart hoặc toàn bộ config migration. Kết quả và giới hạn.

Ví dụ đã chạy: deny chung, allow riêng cho Python

Trích nguyên trường permission của hai case `overlap-allow` trong raw 10 CLI cases, không phải config production đầy đủ. Đây là đối chiếu hai cấu hình đã thử, không phải bằng chứng converter tự động:

```
{
  "permission": {
    "*": "deny",
    "read": "allow",
    "edit": "allow",
    "glob": "allow",
    "grep": "allow",
    "list": "allow",
    "bash": { "*": "deny", "python3 *": "allow" }
  }
}
```

```
{
  "permissions": [
    { "action": "*", "resource": "*", "effect": "deny" },
    { "action": "shell", "resource": "*", "effect": "deny" },
    { "action": "shell", "resource": "python3 *", "effect": "allow" }
  ]
}
```

Fixture yêu cầu Python ghi file witness `REPORT_PERMISSION_EXECUTED`; không thử lệnh xóa nguy hiểm. Các quyền read/edit khác không được đối chiếu tương đương từ ví dụ shell này. Code cấu hình và witness, audit review.

Case	Mong đợi	Thực tế đã đo
CLI allow, v1/v2	Shell thực thi	Có witness ở cả hai
CLI deny, v1/v2	Không thực thi	Không witness ở cả hai
CLI overlap-allow, v1/v2	Exception `python3 *` được allow	Có witness ở cả hai
CLI overlap-deny, v1/v2	Exception `python3 *` bị deny	Không witness ở cả hai
CLI ask unattended, v1/v2	Không tự approve	Không witness ở cả hai
API v1 once/reject/always	Witness theo quyết định; once hỏi lại	Đạt trong raw sáu turn
API v2 saved scope và configured deny	Saved allow không mở resource khác hoặc vượt configured deny hiệu lực	Đạt trong raw API v2

Ý nghĩa khi migrate: kiểm effective permission và side-effect witness, không chỉ đổi hình dạng config. Legacy permission object đã giữ behavior trong năm case trên v2, nên chuyển sang native ordered rules không phải điều kiện bắt buộc chỉ để sử dụng các case đó.

4.4. Subagent và shell chạy nền

Cả v1 và v2 đều có đường subagent chạy nền trong source tại pin. Không mô tả background subagent là tính năng chỉ có ở v2.

Đường thực thi	OpenCode v1	OpenCode v2
Subagent	`background.start` đăng ký `runTask`; `runInBackground` gọi notify rồi trả trạng thái running/jobId. Foreground chờ completion hoặc promotion; interrupt có đường cancel. E-0109.	`subagents.start` nhận recovery descriptor; background gọi `subagents.background` rồi trả child sessionID. Foreground `jobs.block`; interrupt gọi child interrupt và job cancel. E-0111.
Subagent completion	`notifyBackgroundResult` fork chờ child job, inject completed/error vào parent; status khác trả void trong helper này. E-0110.	`SubagentJob.notify` chờ `jobs.wait` theo child ID rồi gọi `SubagentCompletion.deliver`; helper gửi synthetic message tới parent với child ID/state. notify, deliver. Lineage/completion có scripted runtime case E-0133, không suy exactly-once.
Shell completion	Đường schema shell legacy đã kiểm không có native `background` flag; không đặt subagent notification vào ô này để giả cơ chế shell tương đương. E-0113.	Native shell background đăng ký `notifyWhenDone` và trả shellID/output file. E-0112. Parent tiếp tục trước worker terminal và nhận completion đã kiểm trong scripted demo (E-0132); cần đọc exit/output, không suy success chỉ từ notice state.
Shell native background	Schema legacy shell đã đọc chỉ có `command/timeout/workdir`, không có `background` flag. Không suy mọi plugin/mode hay OS detach đều bị cấm. E-0113.	Shell có nhánh native background, timeout input mặc định 0 ở nhánh này (còn qua prepare), job kèm recovery descriptor; khác việc tự thêm `&` vào command. E-0112.

V2 `pendingBackground` đọc persisted `JobTable`, trong khi `runningBackground` lọc live jobs từ state trong bộ nhớ (E-0117). **Có recovery metadata/row không đồng nghĩa process hoặc model/tool execution sống nguyên trạng qua restart.**

Đối chiếu v1 đúng đường legacy: wrapper dùng `InstanceState` (E-0146); core registry khởi tạo `SynchronizedRef(new Map())` và scope (E-0147). Đây không phải persisted job table của v2, nhưng không đủ kết luận mọi v1 host/plugin không có recovery hoặc session history không persist.

Recovery consumer theo source; runtime bổ sung bên dưới:

- Shell job persisted ở trạng thái `running` được chuyển thành notice `cancelled` với nội dung server restarted; `recoverToolJob` gửi synthetic notice `resume: false`, rồi hoàn tất background record. Đây không phải tự chạy lại command hoặc bằng chứng OS process được giữ sống. E-0118.
- Subagent kiểm parent/child còn tồn tại và quan hệ đúng. Job đã kết thúc đi qua completion delivery; child còn running nhưng chưa active có thể được `prepareResume` rồi `execution.resume`, đăng ký lại background job và observer. Đây là resume session execution, **không phải phục hồi instruction pointer hoặc bảo đảm side effect chỉ xảy ra một lần**. E-0119.
- `countResume` tăng durable attempt counter; vượt `maxAttempts` (mặc định 10) phát execution failure và release claim, còn trong budget ghi restart notice. Store giữ `time_suspended/resume_attempts` trong `SessionTable`. Không nhầm budget này với provider retry. E-0120, E-0124.
- Sweep nhận pending jobs, xử lý tool notices trước subagent rồi resume root sessions còn suspended/chưa active. **Activation tùy hosting:** Node boot gọi installer khi có `lifecycle`; `ServerFetch.make` có đường fork recovery riêng. Vì vậy không mặc định mọi `serve`/standalone/SDK có cùng điều kiện kích hoạt. E-0121, E-0122, E-0123.

Acceptance matrix cập nhật tại OFFLINE_DEMO_PLAN.md; test fixture không thay native acceptance. Các lần restart đã thử chỉ trong sandbox/scripted provider, không dùng session thật.

Caller bổ sung: embedded SDK `host.ts` fork sweep trong managed runtime trước owned fetch (E-0145); đây là activation path riêng với Node process/fetch. Không dùng managed-service runtime proof để gọi SDK restart đã được thử.

Runtime: v2 native shell background kiểm success/exit 7, subagent kiểm parent tiếp tục/lineage/completion (E-0132/E-0133); Location-shell cancel/sibling/disconnect là case riêng. V1 background subagent đạt khi bật `OPENCODE_EXPERIMENTAL_BACKGROUND_SUBAGENTS=true`, không phải mặc định (E-0139). Managed-service shell crash đổi PID và có native cancel notice (E-0136); child restart giữ lineage, gọi lại inference và completion (E-0137). Case 11 crashes đạt resume-budget exhaustion, không có request 12; read-only DB xác nhận release/reset (E-0138). Raw budget controller có regex assertion sai được giữ; independent review xác nhận native failure đúng. Stdio cũ không notice vẫn giữ nguyên; không suy exactly-once. Raw và failure lineage.

4.5. Plugin compatibility và port mẫu

Ở pin v1, `PluginInput` có SDK client/project/directory/worktree/serverUrl/BunShell và `Plugin` là function trả `Promise<Hooks>` (E-0104). Entry contract Promise v2 là object `id/setup(context)` với Cleanup và các domain (E-0099). **Bề mặt contract khác nhau không tự chứng minh loader có/không có adapter tương thích cho một module cụ thể.**

Đã truy tới loader, không chỉ so interface:

- **V1:** `applyPlugin` thử `readV1Plugin` rồi gọi `.server(input, options)`; fallback deduplicate các export, nhận function hoặc object có `.server` function. Vì vậy mô tả loader v1 "chỉ nhận default function" là không đúng với đường đã đọc. E-0114.
- **V2:** đường config server module decode schema `default` object có `id` và function `effect` hoặc `setup`; lỗi decode báo yêu cầu definition đó. Nếu là `setup`, `loadServer` dùng `PluginPromise.fromPromise`. **Default function v1 nguyên dạng không thỏa schema ở đường nạp này**; không mở rộng kết luận sang mọi integration path. E-0115.
- Adapter Promise v2 gọi `setup(context2)` qua `acquireRelease` và cleanup nếu có; đây là bridge lifecycle Promise/Effect của contract v2, không phải tự chuyển `Promise<Hooks>` v1. E-0116.

Port marker đã kiểm runtime (E-0129–E-0131): plugin v1 load/gọi marker thành công; nguyên bytes trên v2 nhận đúng loader failure; bản port `id/setup/tool.transform` active, gọi marker qua Code Mode `tools.report_marker()` và có cleanup log. Đây là port tool tối giản, không phải full hooks/options/storage compatibility. Chỉ load trong sandbox

riêng, không đổi config/service người dùng. Mẫu, mọi attempt lỗi và kết quả.

Nội dung	Plugin mẫu v1	Plugin port v2
Entry	Async function trả Hooks	Object `id/setup`
Đăng ký tool	`Hooks.tool.report_marker`	`ctx.tool.transform` → `editor.add`
Schema/output mẫu	`args: {}` / string	JSON Schema input / `{ content }`
Lifecycle đã quan sát	Log load	Log load và cleanup
Native test	Load/call trên v1 đạt; nguyên bytes trên v2 bị loader reject	Active và marker call trên v2 đạt

Before, nguyên fixture plugin_v1.js:

```
export default async () => {
  console.error("REPORT_PLUGIN_V1_LOADED")
  return {
    tool: {
      report_marker: {
        description: "Return the fixed research marker; no file, environment, credential or network
access.",
        args: {},
        async execute() {
          return "REPORT_PLUGIN_MARKER"
        },
      },
    },
  }
}
```

After, nguyên fixture plugin_v2.js:

```
export default {
  id: "report.marker",
  async setup(ctx) {
    console.error("REPORT_PLUGIN_V2_LOADED")
    await ctx.tool.transform((editor) => {
      editor.add({
        name: "report_marker",
        description: "Return the fixed research marker; no file, environment, credential or network
access.",
        input: { type: "object", properties: {}, additionalProperties: false },
        async execute() {
          return { content: "REPORT_PLUGIN_MARKER" }
        },
      })
    })
    return () => console.error("REPORT_PLUGIN_V2_CLEANUP")
  },
}
```

Điểm port là entry contract, tool registration, schema/output và cleanup; không chỉ đổi import. Raw nguyên bản bị reject trên v2, raw port active/marker, cleanup log tạo chuỗi kiểm chứng trước → lỗi loader → sau port. Không kết luận mọi v1 plugin đều có cùng khối lượng chuyển đổi.

4.6. Thay đổi khác có bằng chứng

Các khác biệt đã có căn cứ trong phạm vi này là hosting/API envelope ở 4.1–4.2, native permission contract/enforcement ở 4.3, background/recovery ở 4.4 và plugin contract ở 4.5. Không xem chúng là changelog đầy đủ giữa hai release hoặc bằng chứng runtime cho mọi tính năng mới.

5. Thiết kế thực nghiệm

Thiết kế **4 task × 4 CLI × 3 lượt = 48 slot**: B-01 hiểu code và trả `answer.json` theo rubric 10 điểm, B-02 sửa pagination, B-03 thêm TTL/LRU cache, B-04 refactor router nhiều file. Fixture synthetic Python stdlib do nghiên cứu viết, không phải repository lớn hoặc benchmark bên thứ ba. Pin source/binary ở versions.md, fixture/acceptance seal và rubric `b01-evidence-r2` giữ nguyên trước cohort. Protocol r3, launch audit là căn cứ của cohort cuối, không dùng draft hoặc launch cũ làm trạng thái hiện tại.

Thứ tự 48 slot dùng SHA-256 seed cố định, xoay để mỗi CLI xuất hiện ba lần ở mỗi vị trí trên 12 block; không coi đây là randomized trial lớn. Cùng user prompt/fixture/cap/model/provider, nhưng native system prompt/tool schema khác: đo cả hệ **CLI + model**, không tách hiệu ứng nhân quả riêng của runtime. Mỗi lượt có workspace/session mới; hidden tests chỉ supervisor đọc sau khi writer dừng; giới hạn file được sửa và immutable mounts được kiểm.

Giữ `search-r2/paced-900`: concurrency 1, gap ≥ 30 giây xuyên CLI, tối đa 8 upstream/lượt, output ≤ 2048 token (gồm reasoning), socket 45 giây, wall 900 giây từ outer bwrap spawn tới bridge exit, reserve 60 giây sau pacing. Thời gian chính không gồm supervisor grading; native subprocess clock là chỉ số phụ. Guard $16 là estimate, không phải invoice cap. Request bị chặn local không phải upstream error; completion của provider, native CLI và grader được lưu riêng.

Acceptance trước launch: 185 regression tests, 16 native synthetic task × CLI cases và bốn native smoke thật. Synthetic/reference/mutant tests không tính như model giải task; smoke chỉ kiểm tool call + marker, không tính benchmark task score. Cohort cuối có đủ 48 first-attempt slot hợp lệ, không retry hai valid failures. Bootstrap cache/context errors và supplementary smoke retries giữ trong lịch sử, không nhập vào tỷ lệ thành công.

Fairness: cùng cap không đồng nghĩa mỗi workflow chịu ảnh hưởng như nhau. Agent cần nhiều request chịu thêm pacing và dễ chạm budget; context lớn làm token/cost khác dù cùng model. Vì vậy báo đồng thời số request, token/request, pacing và snapshot correctness, không dùng một median để xếp hạng kiến trúc. Định nghĩa và bảng theo từng task.

6. Kết quả benchmark

6.1. Kết quả cohort OpenAI48

48/48 lượt hợp lệ: 46 success (95,83%), 2 failure; không timeout, infra_error hoặc not_run. B-01/B-02/B-03 đều đạt 3/3 success cho từng CLI; B-04: v1 và Pi đạt 2/3, v2 và Codex đạt 3/3. Snapshot/public/hidden pass 48/48; snapshot pass không thay thế end-to-end success.

CLI	Success/12	Failure	Median giây	Mean giây	Upstream	Input tổng	Output tổng
OpenCode v1	11	1	228,15	224,85	80	306.163	13.095
OpenCode v2	12	0	200,20	216,07	78	234.202	13.148
Pi	11	1	164,23	190,51	68	136.527	9.947
Codex CLI	12	0	139,63	153,52	52	638.403	12.624

Median/mean dùng toàn bộ 12 lượt/CLI, gồm failure và pacing. Benchmark chạy ngày 10/11/2026, 17:02:01–19:39:50 UTC (**00:02:01–02:39:50 ngày 11/11/2026, giờ Việt Nam UTC+7**): **2 giờ 37 phút 49 giây**. 278 request benchmark và 11 request smoke đều HTTP 200/completed; 288 gap xuyên hai giai đoạn đạt ≥ 30 giây (min 30,000319 giây). Không

dùng elapsed để suy tốc độ token thuần.

Độ phân tán theo attempt: median/Q1/Q3/min/max của thời gian và input/output token, dẫn xuất từ 48 raw gồm hai failure; quartiles inclusive, không suy significance.

Token và request: Codex dùng tổng input khoảng **4,68 lần Pi**, nhưng có ít upstream hơn (52 so với 68). Trung bình input/request khoảng 12.277 so với 2.008; Codex lớn hơn ở cả bốn task. Chưa có phân rã token theo system prompt/tool schema/history để xác định nguyên nhân. Không quy chênh lệch cho riêng compaction hoặc "context thừa".

Timing: median pacing/lượt của v1/v2/Pi/Codex lần lượt **204,64 / 178,26 / 148,27 / 118,28 giây**; native subprocess lần lượt **227,83 / 199,86 / 163,93 / 139,21 giây**, vẫn gồm các khoảng chờ trong subprocess. Các median tính riêng, không lấy hiệu để gọi là "CLI time thuần". Provider compute/tool execution duration chưa được phân rã đồng nhất. Bảng token/request, cache/request, request/task và định nghĩa timing, script tái tính chỉ đọc.

6.2. Failure và ảnh hưởng của giới hạn

R-0343/v1/B-04 và R-0365/Pi/B-04 có snapshot đúng nhưng ghi native error/local HTTP 400 sau tám upstream completed; v1 exit 1, Pi exit 0. Giữ `failure/valid` ở cả hai. Quan sát phù hợp với chạm cap 8 request; exact local reject reason không được ghi riêng nên không khẳng định một nguyên nhân duy nhất đã đo trực tiếp. Không phải upstream 400 hoặc bằng chứng model viết code sai. Không nới cap/retry để cứu điểm; exit 0 không đủ khi có error event. Raw v1, raw Pi.

Task correctness: cả hai snapshot qua acceptance. **Agent completion:** hai lượt không hoàn tất thành công dưới protocol đã đặt. Vì vậy 11/12 không đồng nghĩa hai tool không sửa được B-04; kết quả phản ánh cả khả năng hoàn tất workflow trong budget. Cả ba lượt Pi/B-04 đều dùng tám upstream nhưng chỉ một lượt failure, nên riêng số request chưa đủ để phân loại.

6.3. Chi phí và bằng chứng

Guard estimate cho 48 lượt khoảng **$2,15**; khoảng tariff của phần usage đã báo là **$1,39–$2,15**. 254/278 request thiếu TTL breakdown được dùng conservative 12h upper estimate, không xác nhận actual TTL/invoice. Reasoning đã nằm trong output; cache writes tách ordinary input để tránh double count. Số chính xác, token/cache từng CLI, 48 rows và raw ở kết quả đầy đủ.

Smoke r3 riêng khoảng $0,10; tổng đợt r3 khoảng $2,24. Cộng guard carry/reserve cũ khoảng **$2,58**, còn khoảng $13,42 so với guard $16. Không gọi đây là tổng charge dự án: sáu request unknown lịch sử vẫn giữ, `measured_charge=null`, pilot Shopaikey có ledger riêng. Không dùng giá OpenAI để tính lại Shopaikey hoặc coi HTTP 400 thiếu usage là miễn phí.

Audit cuối kiểm slot coverage, hashes, source snapshots, caps/usage/pacing, cleanup 52 lượt và prior raw/frozen pilot; registry R378/C479, generation gate đóng, không extra API/key read. Shopaikey benchmark giữ 8 attempts/7 unique slot/41 not_run trong ledger riêng, không gộp vào OpenAI48.

7. Hạn chế

- **Ngoại suy:** bốn fixture synthetic Python nhỏ, ba lượt/task và 12 lượt/CLI; chưa đánh giá đầy đủ repository lớn, đa ngôn ngữ, long-context hoặc chất lượng production. Không khẳng định significance/winner tổng quát từ mẫu này.
- **Confounding:** cùng model/provider nhưng native prompt/tool schema/bootstrap/số bước khác. Pacing ≥ 30 giây, cap 8 request và cache ảnh hưởng timing/cost/completion; cache state/backend revision/background load chưa được kiểm soát hoàn toàn. Không suy "bản mới nhanh hơn" theo quan hệ nhân quả chỉ từ median.
- **Chi phí:** conservative TTL upper bounds chỉ là tariff estimate cho usage đã báo, không gồm phí không báo. Unknown usage lịch sử không bằng zero; hóa đơn/account balance chưa đối soát. Cùng alias trên gateway không chứng minh cùng backend OpenAI.

- **Bảo mật/lifecycle:** bwrap của harness không chứng minh native sandbox. Demo native dùng scripted local inference, không model quality/semantic memory. Stdio cũ không notice không bị nâng thành pass; managed-service đạt sau sửa boot-readiness của harness. Linux headless/Chromium web engine/PTY không đại diện mọi display/OS; v1 approval qua API không phải keyboard UI. Không suy exactly-once hoặc exact SDK build → source attestation; installed-tree hash không phải lockfile.
- **Lịch sử/classification:** giữ bootstrap/cache/adapter errors, không tính như model task failures; hai end-to-end failures vẫn giữ, không nâng snapshot pass thành success. Bằng chứng đã xóa theo yêu cầu trước đây không được phục hồi hoặc dùng lại.

8. Kết luận và khuyến nghị

Trong phạm vi nghiên cứu, source tại release pins cho thấy khác biệt về hosting, agent loop/tool, policy/context và extension; OpenAI48 cung cấp số đo end-to-end dưới một protocol chung. v2/Codex hoàn tất 12/12, v1/Pi đạt 11/12; tất cả snapshot đạt. Codex có median thấp nhất nhưng guard estimate cao nhất, Pi có estimate thấp nhất. Chưa đủ căn cứ tuyên bố một tool tốt nhất hoặc provider ổn định nhất.

Tình huống	Công cụ nên đánh giá trước	Căn cứ và trade-off
Nhiều client dùng chung session hoặc automation qua service	OpenCode v2	Có Client/service và demo shared UI; cần chọn đúng managed/standalone/embedded lifecycle (3.1, 4.1–4.2), không xem mọi mode là cùng process
Agent cần tùy biến tools và hành vi theo event	Pi	Tách Agent/AgentSession, có extension callbacks và tool registration (3.1, 3.4, 3.6); hook không thay OS isolation, extension runtime chưa demo trong nghiên cứu
Coding workflow CLI với approval và sandbox backend	Codex CLI	Nhánh exec/app-server, router và approval/sandbox tách biệt (3.1, 3.3–3.4); cần kiểm backend thực sự active trên OS mục tiêu

Đây là ưu tiên đánh giá tích hợp, không phải bảng xếp hạng sản phẩm. Benchmark đối chiếu thêm trade-off: Codex ít request/median thấp hơn nhưng input và estimate cao hơn; Pi estimate thấp nhưng B-04 chịu nhiều bước/pacing. Chọn theo workload và bề mặt tích hợp, không chỉ theo tổng token hoặc tốc độ ở bốn fixture nhỏ. Khi migrate v1 → v2, kiểm effective config/permission, API envelope và plugin loader riêng; không chỉ đổi URL hoặc import type.

Vận hành benchmark: giữ quota/budget guard và failure raw; log rõ local reject reason; phân biệt snapshot quality với terminal completion; giữ reasoning fields theo native protocol và định giá cache writes đúng. Không đổi caps sau khi thấy result để sửa thứ hạng. Thử nghiệm bổ sung cần scope/ngân sách riêng; phần unverified không được biến thành kết quả đạt.

9. Phụ lục và bằng chứng

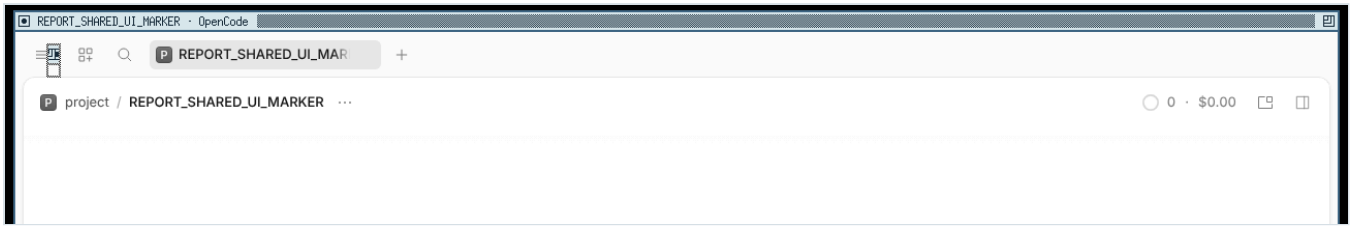
Bộ nộp không chỉ gồm `REPORT.md`. Repository public: binhdinhvan/agent-cli-report giữ cấu trúc đường dẫn tương đối để người review đi từ nhận định → code/source → raw → audit. Hướng dẫn đọc và điều kiện tái chạy, phạm vi công bố và raw gzip lịch sử. Credentials/binary/dependencies cục bộ không được đưa lên GitHub; công bố không thay thế các giới hạn kiểm chứng đã nêu.

Yêu cầu	Vị trí	Bằng chứng chính
So sánh ba tool: kiến trúc, agent loop, tool	3.1–3.3	Source permalinks cố định trong từng bảng; evidence index
Permission/sandbox, context, extensibility	3.4–3.6	Source permalinks; không nâng source-only thành runtime
Cài song song v1/v2 đúng ngày chốt	1–2	Versions/full SHA, binary/checksum/isolation
Shared background service	4.1	UI05 ID/PID, ảnh và raw PTY ở 4.1
Script API/SDK	4.2.1	Client code, Client raw, SDK code
Permission object/rules	4.3	10 CLI cases, API/legacy cases liên kết tại mục
Background subagent/shell	4.4	Hai vòng demo, raw recovery/budget và giới hạn
Plugin port	4.5	Before/after fixtures, loader failure, marker và cleanup log liên kết tại mục
Benchmark success/token/time	5–6	Protocol, CSV 48 lượt, raw/audit

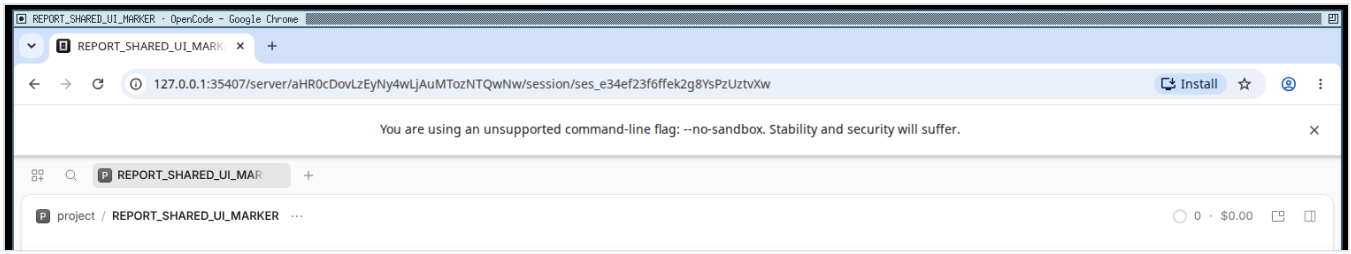
Chi tiết acceptance theo tám hạng mục gốc: REQUIREMENTS_ACCEPTANCE.md. Phân tích sâu: fairness/token/timing. Lịch sử provider/protocol/retry và dữ liệu đã rút: EXPERIMENT_HISTORY.md.

Một session, nhiều giao diện

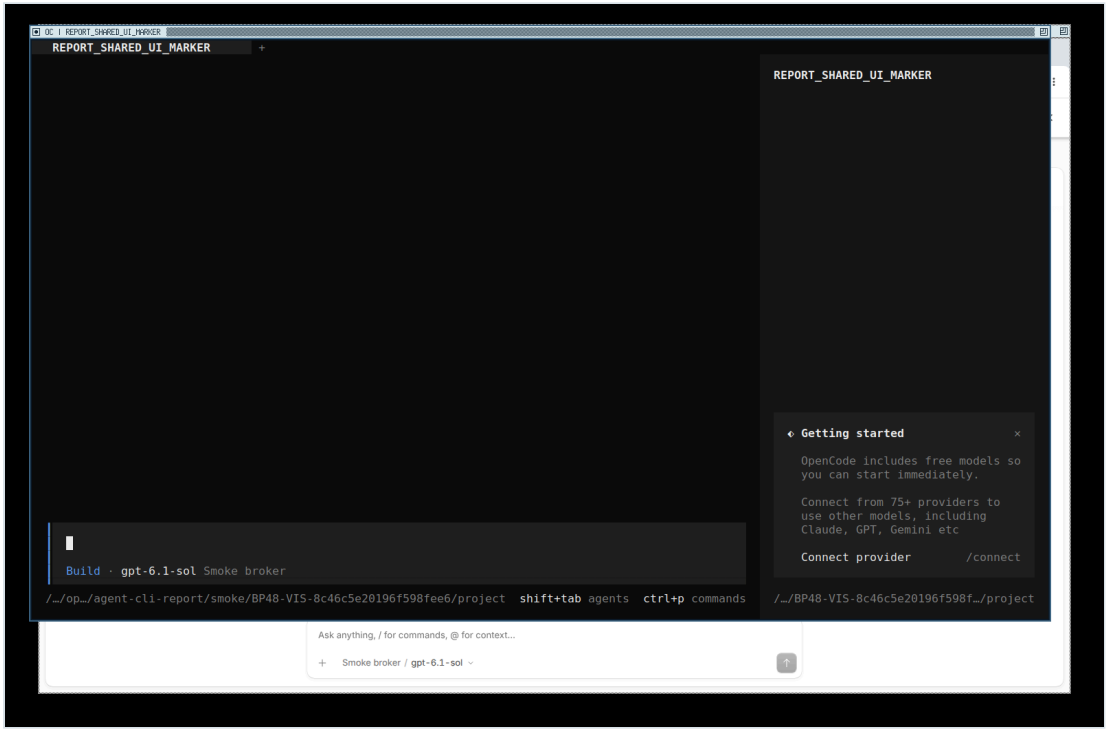
Lượt chụp mới: VISIBLE-UI07 · Desktop 2.2.5, Chrome độc lập và TUI trên Xvfb riêng. Không gọi model.



A1. Desktop đóng gói · cắt vùng trên để thấy rõ title bar thật và session title. Renderer `oc://renderer/index.html`.



A2. Web trên Chrome độc lập · cắt vùng trên có toolbar/thanh địa chỉ thật, URL chứa đúng session ID. Không vẽ thêm browser chrome.



A3. TUI native trong xterm · cùng title; command/raw PTY đối chiếu đúng session ID. Ảnh gốc không có chữ hoặc khung dựng thêm.

SHARED SERVICE VERIFICATION · ĐỐI CHIẾU RAW

Session ID: `ses_e34ef23f6ffek2g8YsPzUztvXw`

Desktop: cùng session · PASS **Web: cùng session · PASS** **TUI: cùng session · PASS**

Service PID trước: **12** | sau cả ba client: **12** | **Shared-service checks: PASS**

Endpoint: `http://127.0.0.1:35407`. Raw controller: `checks_passed_pending_review`; review ảnh/raw riêng xác nhận PASS.

Review ảnh/raw riêng — `status=pass`:
`research/artifacts/NATIVE-VISIBLE-UI-REVIEW-20261111-01/validation.json`
Receipt đối chiếu ảnh, session/PID, hashes và cleanup; raw controller giữ nguyên trạng thái pending review.

Ảnh gốc 1600 × 1050 giữ riêng; A1/A2 chỉ cắt pixel để phóng chi tiết, không sửa nội dung. Cảnh báo Chrome `--no-sandbox` giữ nguyên; có bwrap bên ngoài, không phải security verdict. PID chỉ có ý nghĩa trong namespace này.

Nguồn cùng lượt: `raw_ID/PID/checks` · `ảnh_Desktop_gốc` · `ảnh_Chrome_gốc` · `ảnh_TUI_gốc` · `raw_PTY`.
Permalink bằng chứng cố định theo commit `f2becc501cfc`.

API, permission và plugin port

Trích trường/log từ artifact thật, trình bày lại để đọc nhanh — không phải screenshot terminal hoặc lần chạy mới. Native runtime với scripted local SSE, không đo model quality.

B1. Client / Embedded SDK

Client tạo session, gửi `REPORT_CLIENT_USER_SEED`, nhận assistant marker và terminal event:

```
{
  "type": "session.execution.succeeded",
  "created": 1794352917765,
  "data": {
    "sessionID": "ses_e383b54d3ffesBFcD9WgE4b3Y8"
  }
}
```

`subscribed_before_prompt=true`; `transcript_after_restart=true`.
SDK riêng: `embedded_prompt_assistant_marker=true` và `explicit_host_close_returned=true`.

Nguồn: demos/artifacts/NATIVE-CLIENT-20261111-01/client-result.json

Nguồn: demos/artifacts/NATIVE-SDK-20261111-01/sdk-result.json

B2. Permission v1 / v2

Rule / case	v1: thực tế	v2: thực tế
`allow`	Có witness	Có witness
`deny`	Không witness	Không witness
`ask`	Không witness	Không witness

Witness là file do shell fixture ghi. `ask` ở bảng là CLI unattended: không tự approve, không phải người dùng bấm từ chối trong UI. API once/reject/always có case riêng trong báo cáo.

Nguồn: demos/artifacts/NATIVE-DEMO-PERMISSION-20261111-02/summary.json

B3. Plugin v1 → v2

Nguyên bytes v1 trên v2: loader báo lỗi. Sau port `id/setup/tool.transform`: active, gọi `report_marker` thành công.

Before port — nguyên plugin v1 nạp trên v2:

```
Plugin must export a default definition with an id and an effect or setup function.
```

After port — lượt chạy bản v2 riêng:

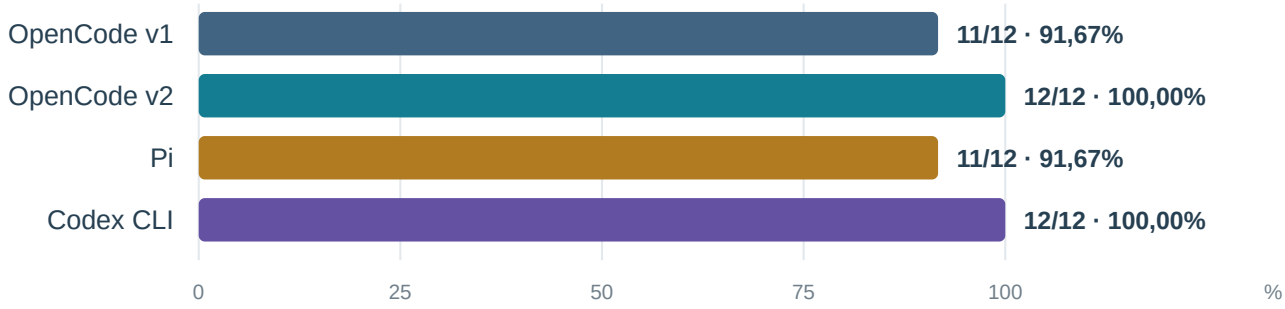
```
ported_plugin_active: true
ported_plugin_tool_invoked: true
REPORT_PLUGIN_V2_LOADED
REPORT_PLUGIN_V2_CLEANUP
```

Nguồn: demos/artifacts/NATIVE-CLIENT-V1PLUGIN-20261111-02/client-result.json
demos/artifacts/NATIVE-PLUGIN-PORT-20261111-03/client-result.json
demos/artifacts/NATIVE-PLUGIN-PORT-20261111-03/client-server.log

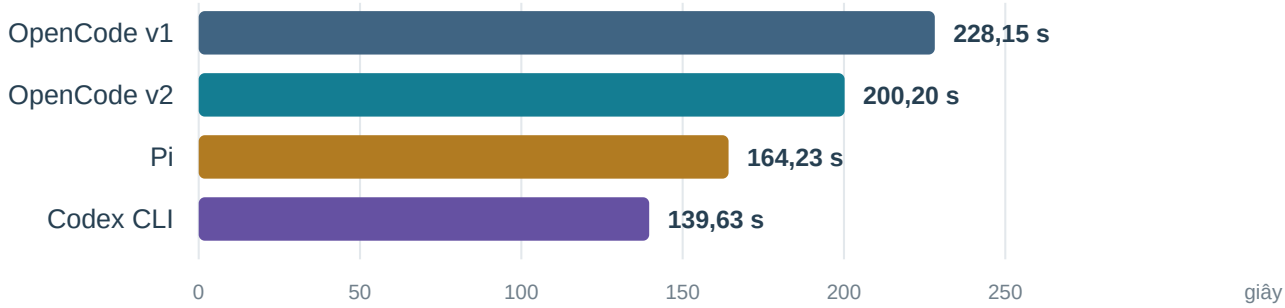
Ba góc nhìn, cùng một cohort

4 task × 4 CLI × 3 lượt · cùng model/provider trong cohort · v1.19.5 / v2.2.5 / Pi 1.5.0 / Codex 0.167.0.

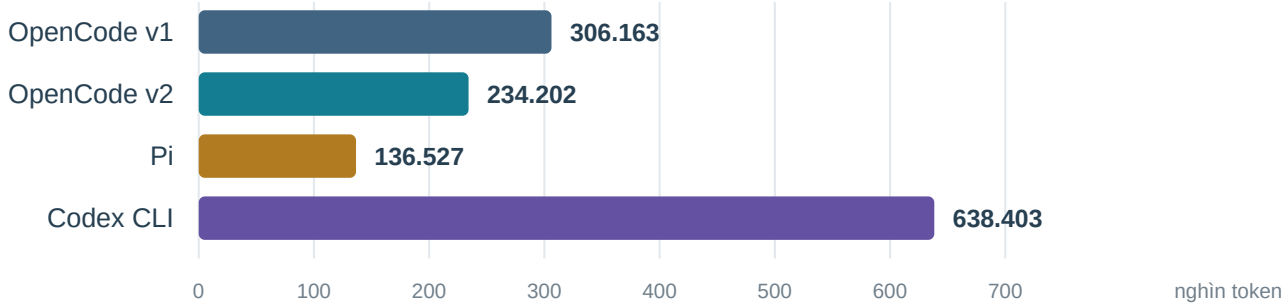
C1 · Hoàn tất end-to-end



C2 · Median thời gian / lượt



C3 · Tổng input token / 12 lượt



Cách đọc: completion là agent hoàn tất theo protocol, không chỉ snapshot đúng (48/48 snapshot pass). Median dùng cả 12 lượt/CLI, gồm failure, startup/teardown và **pacing ≥ 30 giây**. Token là usage input tích lũy, có cache; không phải context duy nhất hoặc hóa đơn. Không suy winner tổng quát từ bốn fixture Python nhỏ.

Nguồn: `benchmark/suite48/artifacts/OPENAI48-R3-SUMMARY-20261111-01/runs48.csv`

Input token được cộng từ `48_raw_result.json` → `requests[].cost.usage.input_tokens`. Vẽ biểu đồ từ dữ liệu gốc, không dùng số đã làm tròn để tính median.